



Releasing

Given a scenario, analyze and explain the use cases and considerations when using managed, unmanaged, and unlocked packages.

Develop your career with Salesforce Training and Certification Preparation

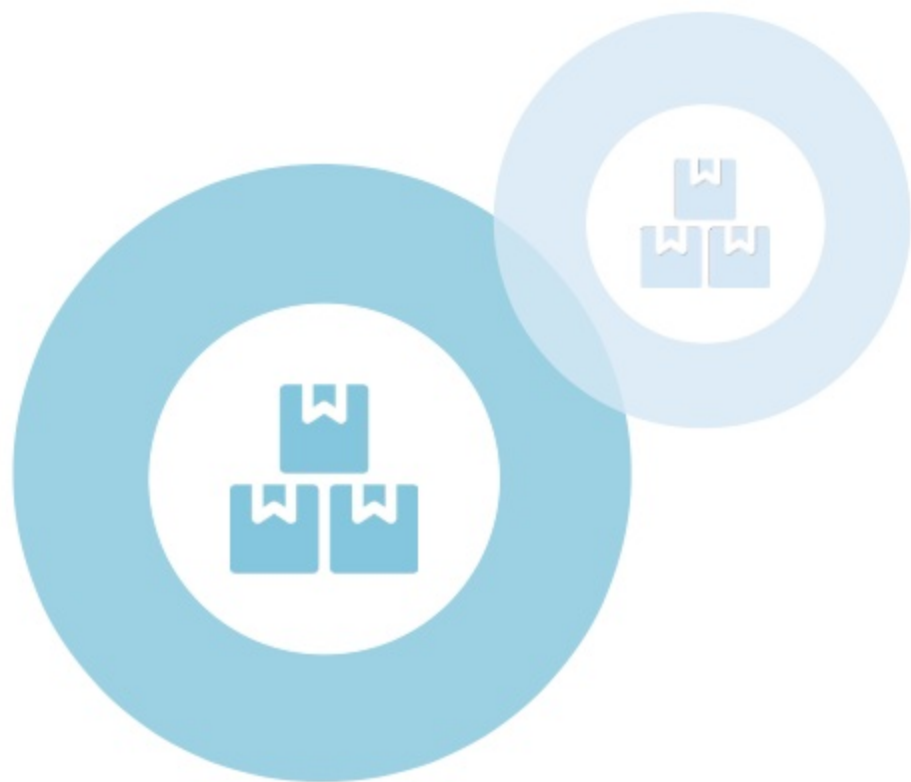
Table of Contents

-  [Managed Packages](#)
-  [Unmanaged Packages](#)
-  [Unlocked Packages](#)
-  [Business Scenario](#)
-  [Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Describe the use cases and considerations of managed packages.
-  Describe the use cases and considerations of unmanaged packages.
-  Describe the use cases and considerations of unlocked packages.
-  Explain the use cases and considerations when using a managed, unmanaged, or unlocked package for a given business requirement.



Introduction

In the Salesforce ecosystem, the deployment and distribution of custom applications and functionalities can be effectively managed through different types of packages: **managed, unmanaged, and unlocked packages**. Each package type serves distinct use cases, from distributing applications on the AppExchange, sharing open-source projects, to deploying modular customizations within an organization.

Managed packages are ideal for ISVs to **distribute and sell applications** with upgrade paths and license management. **Unmanaged** packages are suitable for **one-time distributions** where the recipient may need to modify the source code. **Unlocked packages** provide more flexibility for **iterative development** and **release management** within **Salesforce DX**, offering a balance between controlled deployment and customization capability. Understanding when and how to use each package type is crucial for Salesforce developers and architects to ensure efficient, secure, and scalable application lifecycle management.



Managed, Unmanaged, and Unlocked Packages

Managed Package Use Cases

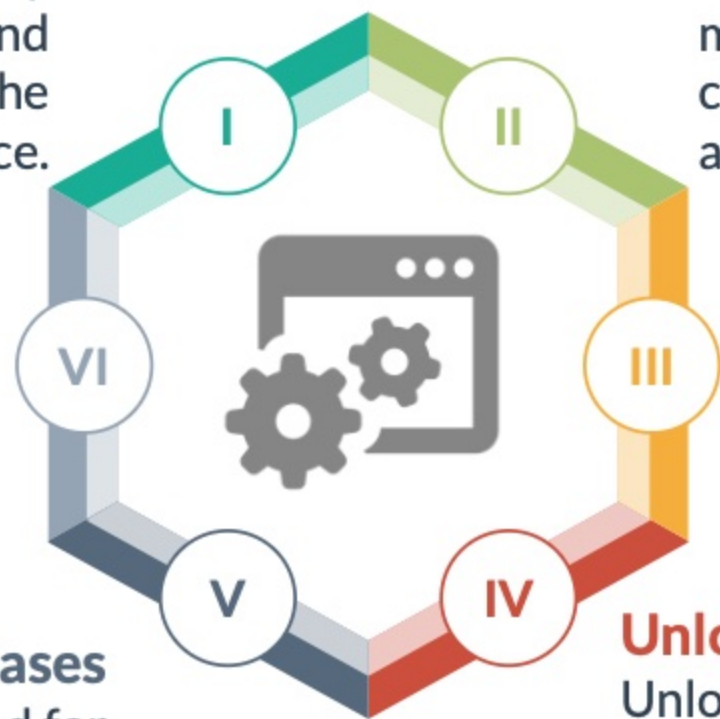
Managed packages are used by Salesforce partners to create apps and distribute them to customers via the AppExchange marketplace.

Unmanaged Package Use Cases

Unmanaged packages are used to distribute open-source projects to developers or for one-time upload of applications that require customization after installation.

Unlocked Package Use Cases

Unlocked packages are designed for customers and partners to organize and deploy metadata and business logic using modular development.



Managed Package Considerations

The same namespace can be assigned to multiple managed packages. Public Apex classes and methods can be shared across packages in the same namespace.

Unmanaged Package Considerations

The components installed from an unmanaged package can be edited. Components can be added either explicitly or implicitly to an unmanaged package.

Unlocked Package Considerations

Unlocked packages utilize source-driven development, allow declaring multi-level dependencies, and can include a namespace for isolation or be namespace-free.

Managed Packages

[Table of Contents](#) 

Managed Packages

Managed packages are used by **Salesforce partners** to **create apps** and **distribute** them to customers via the **AppExchange** marketplace.

❖ CONTAINER

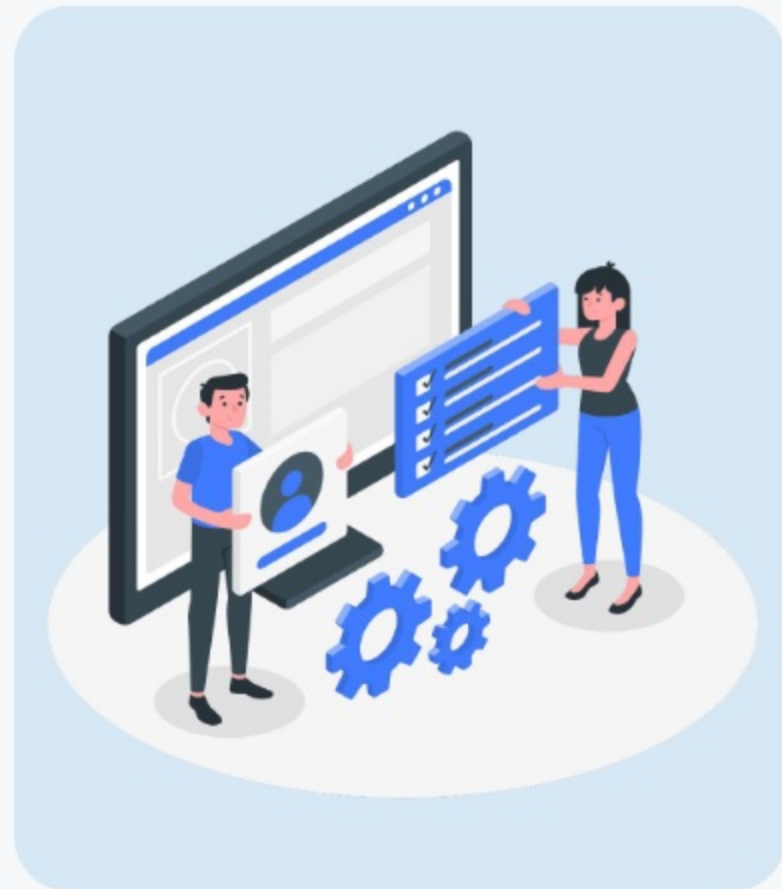
A package is a container that holds a set of **related features**, **customizations**, and schema. It can include many different **metadata components** and allows packaging a single **component**, **app**, or **library**.

❖ CAPABILITIES

Managed packages allow partners to **distribute**, **license**, **pilot features of**, **troubleshoot**, and **monetize** their offerings.

❖ TYPES

Salesforce offers **first-generation managed packages (managed 1GP)** and **second-generation managed packages (managed 2GP)**. Going forward, Salesforce recommends the use of **managed 2GP**.



Managed Packages

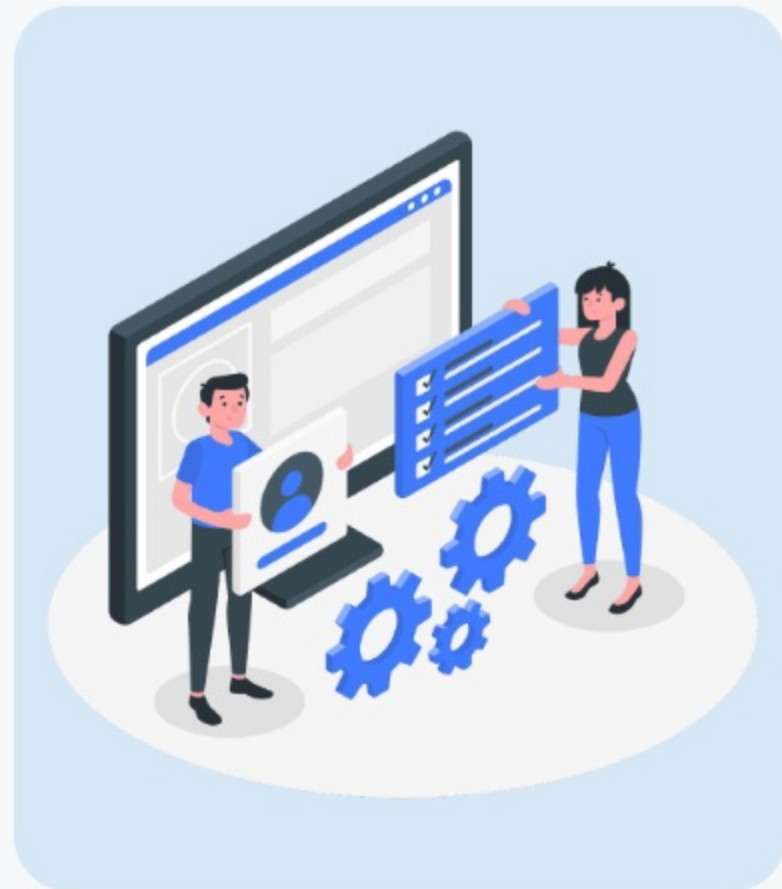
Managed packages are used by **Salesforce partners** to **create apps** and **distribute** them to customers via the **AppExchange** marketplace.

❖ MANAGED 2GP

Managed 2GP offers benefits such as **source-driven development**, **minimal interaction** with Salesforce orgs, **API-** and **CLI-first** model, **flexible versioning**, **namespace sharing** across packages, **declarative dependencies**, and **simplified patch versioning**.

❖ LIFECYCLE

Each package follows a **distinct** lifecycle. Whenever **metadata** is changed, added, or removed in a package, a **new package version** is created. Each package version has a version number and is an **immutable** artifact. When a new version is released, customers can **upgrade** to the latest package version or an upgrade can be **pushed** to them.



Managed Packages

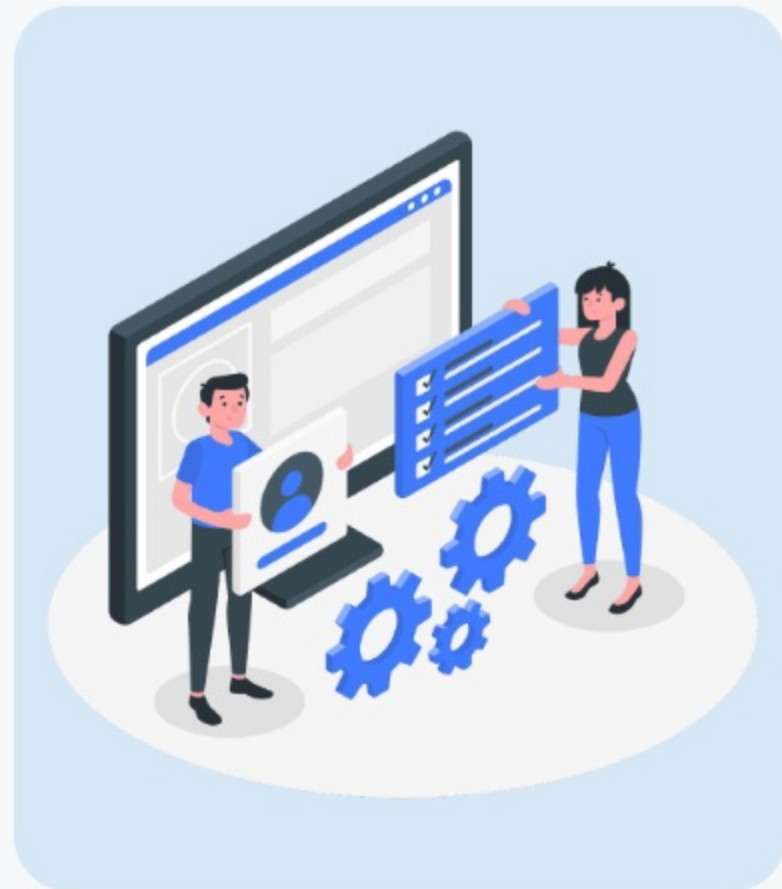
Managed packages are used by **Salesforce partners** to **create apps** and **distribute** them to customers via the **AppExchange** marketplace.

❖ METADATA

Features, customizations, and schema that comprise an app can be added to a package. Examples of **metadata components** include Apex classes, custom objects, flows, etc. The **Metadata Coverage Report** can be viewed for a complete list of metadata components that can be added.

❖ APPEXCHANGE

When a **completed app** has been packaged up, and the **packaged app** has been through the **security review**, it can be listed on the **AppExchange** for **marketing** and **distribution**.



Managed Packages

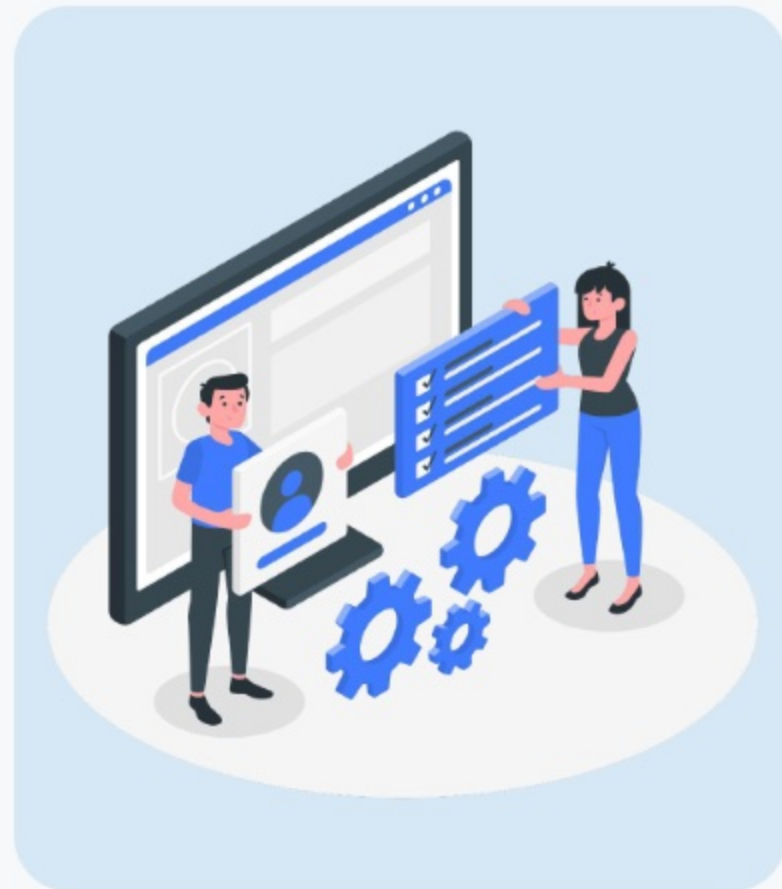
Managed packages are used by **Salesforce partners** to **create apps** and **distribute** them to customers via the **AppExchange** marketplace.

❖ NAMESPACE

A namespace is an **alphanumeric identifier** that can be created, registered, and associated with one or more packages. The **same namespace** can be assigned to **multiple managed 2GPs**. **Code** can be **easily shared** across packages in the same namespace.

❖ CONTINUOUS INTEGRATION

End-to-end automation can be created using **continuous integration (CI) jobs**. For example, a CI job can be launched when a **pull request** against the **source repository** is created. The job can **create a package version** and **a scratch org**, and then **install** the package in the org.



Managed Packages

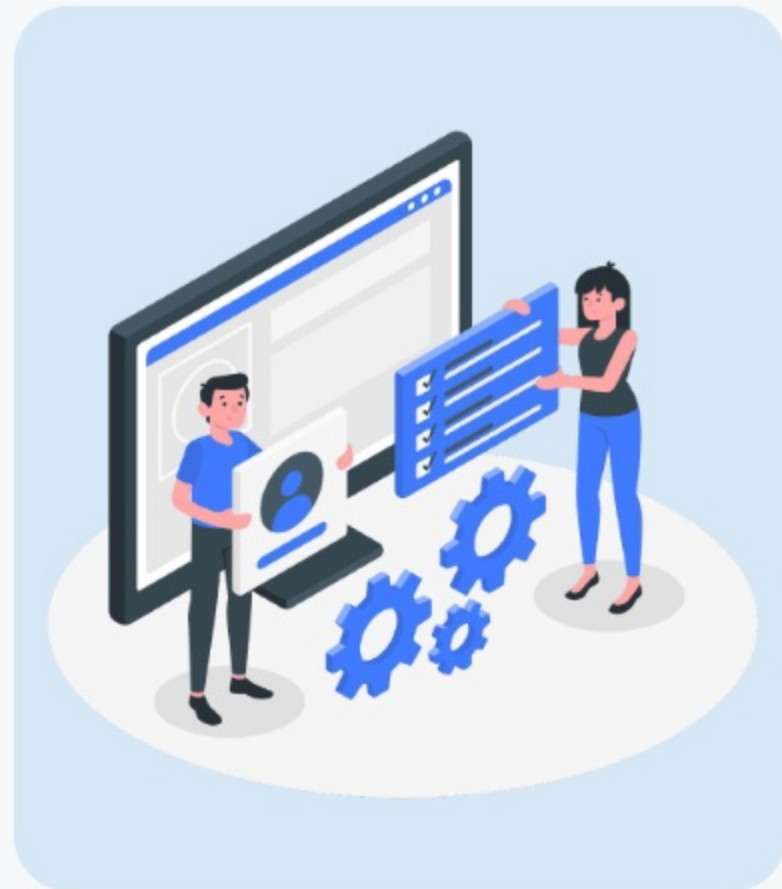
Managed packages are used by **Salesforce partners** to **create apps** and **distribute** them to customers via the **AppExchange** marketplace.

❖ CODE SHARING

Public Apex classes and methods can be **shared** across **all packages in the namespace**. The **@namespaceAccessible** annotation can be added to an **Apex class**, making it available to all packages in the namespace.

❖ FLEXIBLE VERSIONING

An **ancestor** needs to be specified for **each new package version**. A package version can be **abandoned** if it does not need to be built on, but only if **customers haven't been upgraded** to it yet. Moreover, when **creating a package version**, it can be **tagged with the name of the branch** in the source control system.



Managed Packages

Managed packages are used by **Salesforce partners** to **create apps** and **distribute** them to customers via the **AppExchange** marketplace.

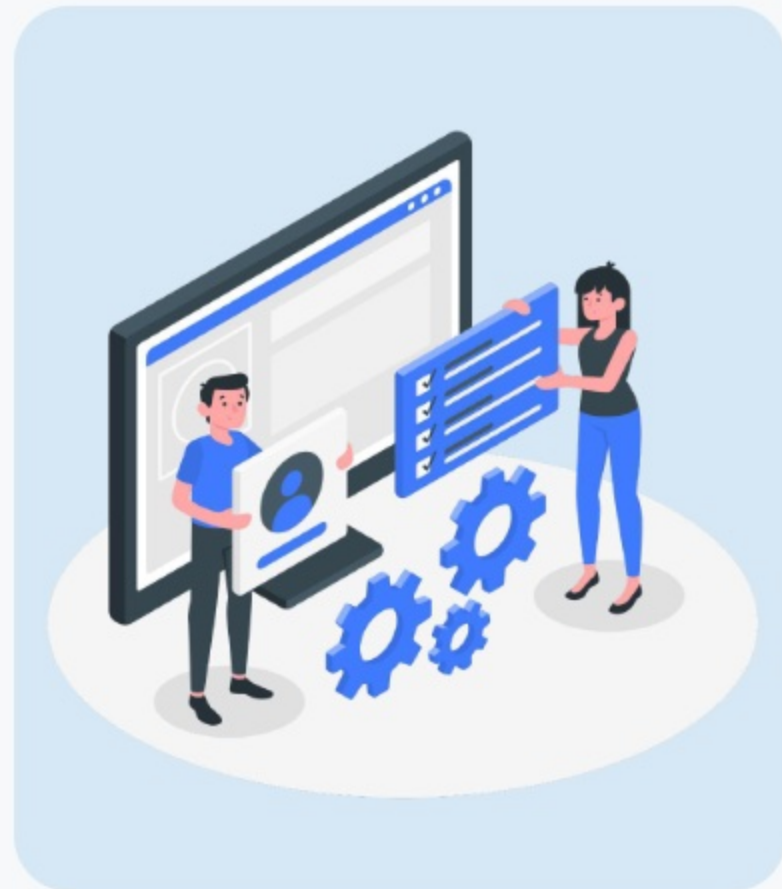
❖ PARALLEL DEVELOPMENT

Two or more development teams can build new features in **parallel** for the **same package** and then **merge** them when appropriate. Features can be built and tested without taking over the main branch. Unreleased versions can be easily abandoned.

❖ TOOLS

The **key tools** used for building a managed 2GP are as follows:

- ❖ Salesforce Extensions for VS Code
- ❖ Salesforce CLI
- ❖ Dev Hub org
- ❖ Source Control System



Managed Packages

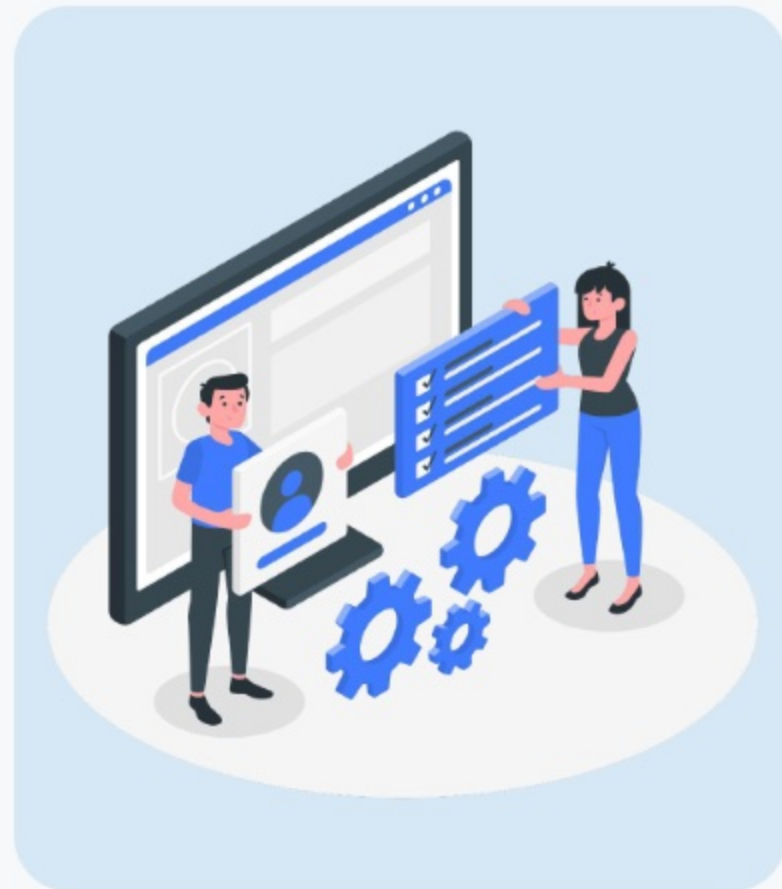
Managed packages are used by **Salesforce partners** to **create apps** and **distribute** them to customers via the **AppExchange** marketplace.

❖ DEV HUB

The **Dev Hub** setting can be enabled for a Salesforce org to make it the **Dev Hub org**, which is the designated place to find and manage all the **managed 2GP packages**, **scratch orgs**, and **namespaces**.

❖ PACKAGE CONFIGURATION

A **Salesforce DX project directory** contains **one or more package directories**. The **sfdx-project.json** configuration file describes the configuration for each package. It specifies information such as **package namespace**, **API version**, and the **path**, **version name**, **description**, **definition file**, and **dependencies** of each package.



Managed Packages

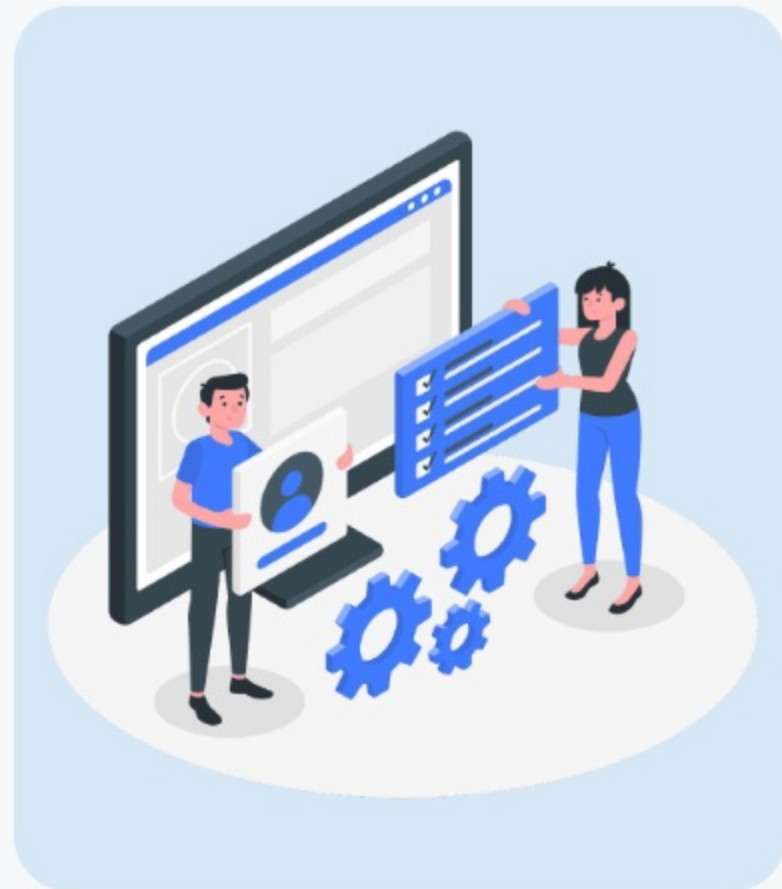
Managed packages are used by **Salesforce partners** to **create apps** and **distribute** them to customers via the **AppExchange** marketplace.

❖ VERSION NUMBERING

Package version numbers use the **semantic versioning** format. Each package version has **major version**, **minor version**, **patch version**, and **build number** (e.g., 4.1.7.NEXT). The build number can be **NEXT**, **LATEST**, or **RELEASED**.

❖ SCRIPTS

Continuous integration **scripts** can **automate package development steps**. For example, a script can be written to create a package version, create a scratch org, install the package version in the scratch org, run Apex tests, and email the test results to the release manager.



Managed Packages

Managed packages are used by **Salesforce partners** to **create apps** and **distribute** them to customers via the **AppExchange** marketplace.

❖ CUSTOMIZATION

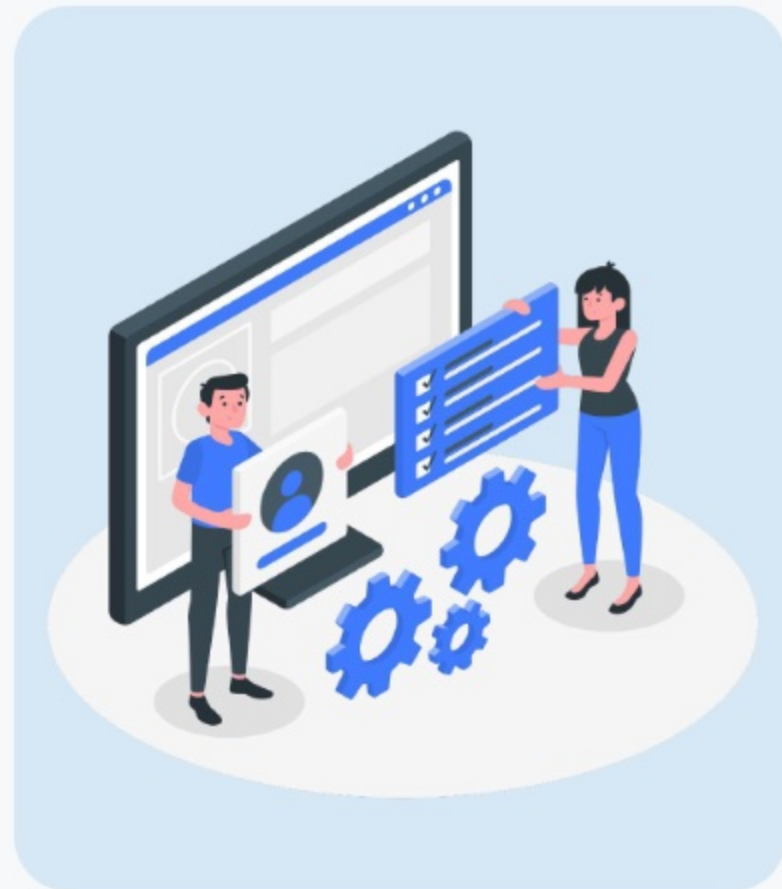
Each **metadata component** in a package has **certain rules** that determine whether the **developer** or the **subscriber** can **make changes** to the component or **remove** it after the package version is installed.

❖ APEX SETTINGS

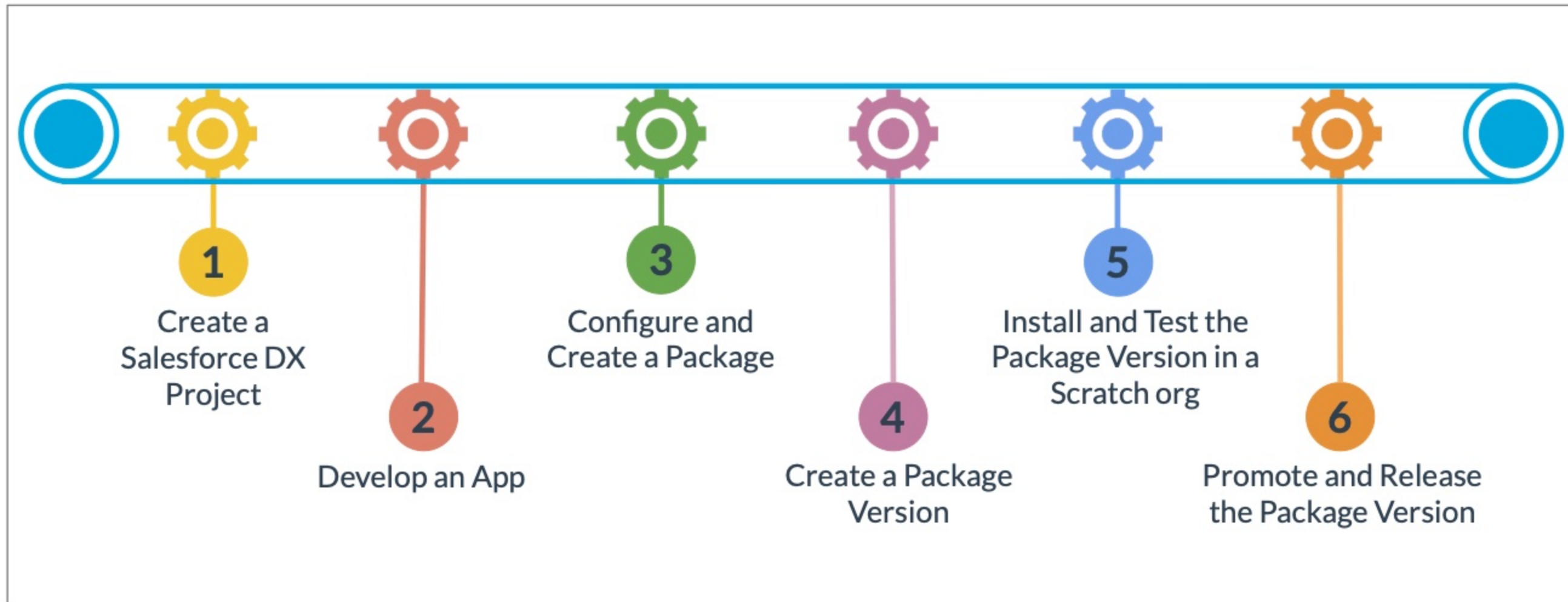
To enable deployment of metadata from Apex code in a package that is not yet certified, **Deploy Metadata from Non-Certified Package Versions via Apex** can be enabled on the **Apex Settings** page in Setup.

❖ DEVELOPER EDITION ORGS

A partner can use a **Developer Edition** or **Partner Developer Edition** org for the development of a managed package.



Managed 2GP Development Workflow



Unmanaged Packages

[Table of Contents](#)



Unmanaged Packages

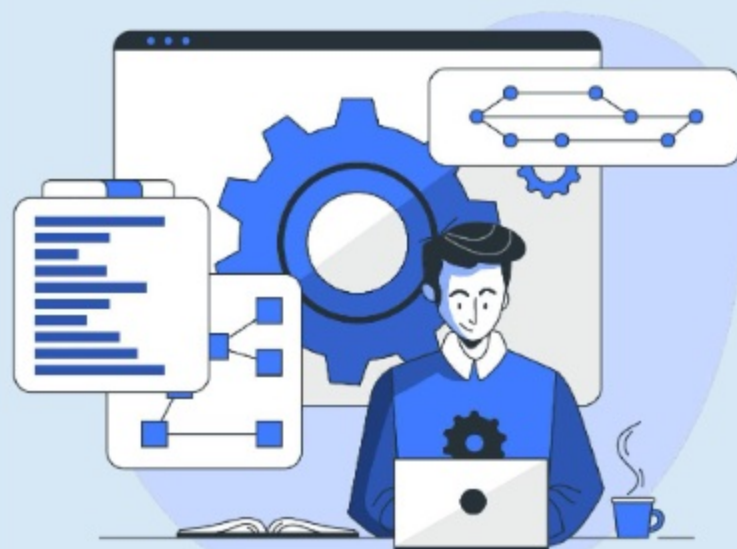
Unmanaged packages are used to **distribute open-source projects** to developers or for **one-time upload of applications** that require customization after installation.

❖ CUSTOMIZATION

The **components** installed from an unmanaged package can be **edited** in the org they're installed in. The developer who **created and uploaded** the package has no control and **cannot change or upgrade** them.

❖ BEST PRACTICE

An unmanaged package should **not be used to migrate metadata** from a sandbox to production. Furthermore, it should **only be installed** if the org used to upload the package still exists.



Unmanaged Packages

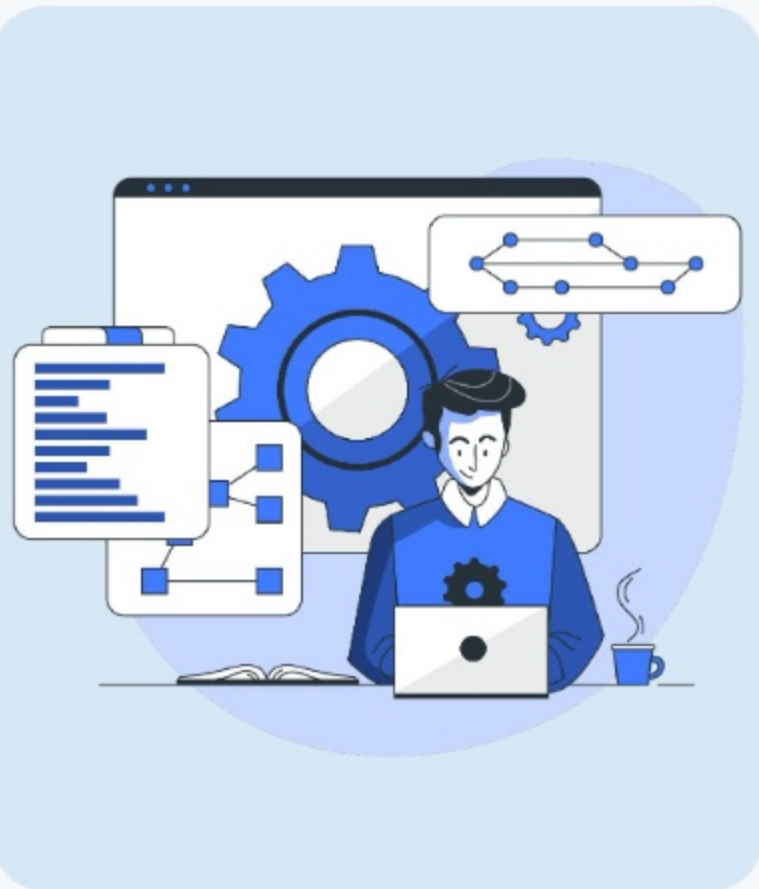
Unmanaged packages are used to **distribute open-source projects** to developers or for **one-time upload of applications** that require customization after installation.

❖ CREATION

An unmanaged package can be created and uploaded on the **Package Manager** page in Setup. The **Components** tab can be used to add components to the package.

❖ UPLOAD

Once all the components have been added to an unmanaged package, it can be **uploaded**. An **email with an installation link** is sent once the package is uploaded successfully.



Unmanaged Packages

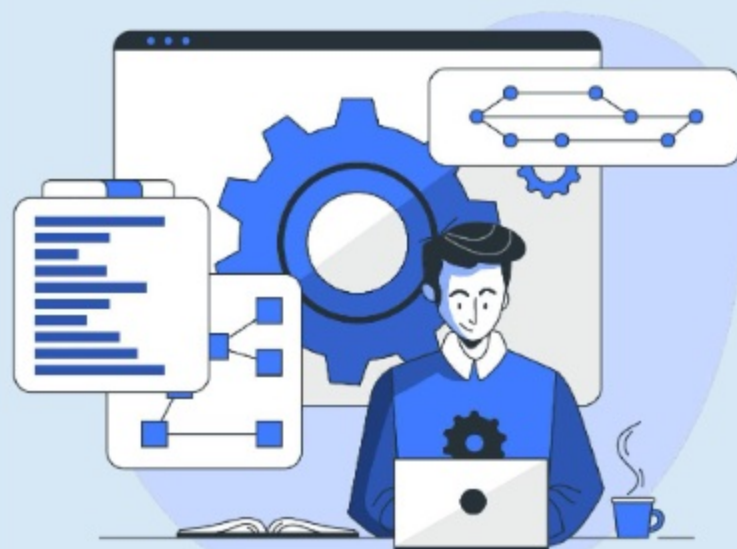
Unmanaged packages are used to **distribute open-source projects** to developers or for **one-time upload of applications** that require customization after installation.

❖ UNINSTALLATION

Any **dependencies on metadata** in another package must be **removed** before attempting to **uninstall an unmanaged package**.

❖ COMPONENTS

Components can be added **either explicitly** or **implicitly** to an unmanaged package. Explicit components must be **included directly** in the package, while implicit components are **added automatically** when another dependent component, such as a custom object, is added.



Unmanaged Packages

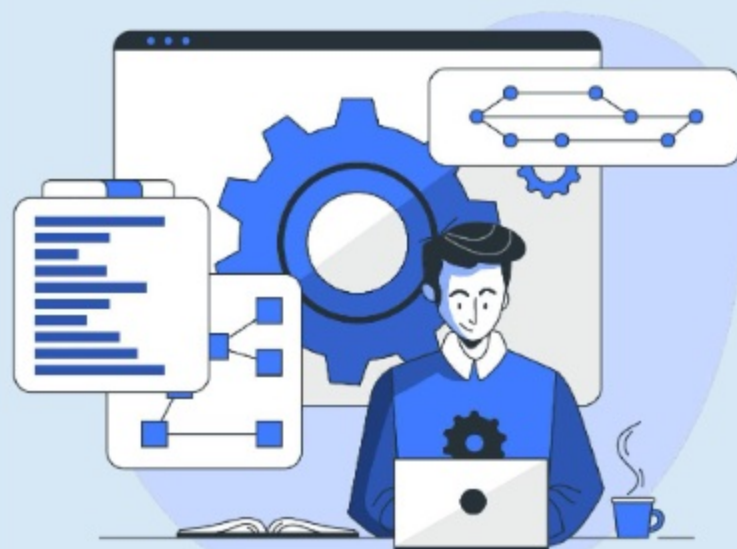
Unmanaged packages are used to **distribute open-source projects** to developers or for **one-time upload of applications** that require customization after installation.

❖ AUTOMATIC RENAMING

Some components, such as Report and Dashboard, support **automatic renaming** if a **naming conflict** occurs during installation. For other components, the installation is **blocked** if a naming conflict occurs.

❖ CONVERSION

An **unmanaged package** can be **converted** into a **managed package** on the **Package Manager** page in Setup after **registering a namespace**.



Unlocked Packages

[Table of Contents](#)



Unlocked Packages

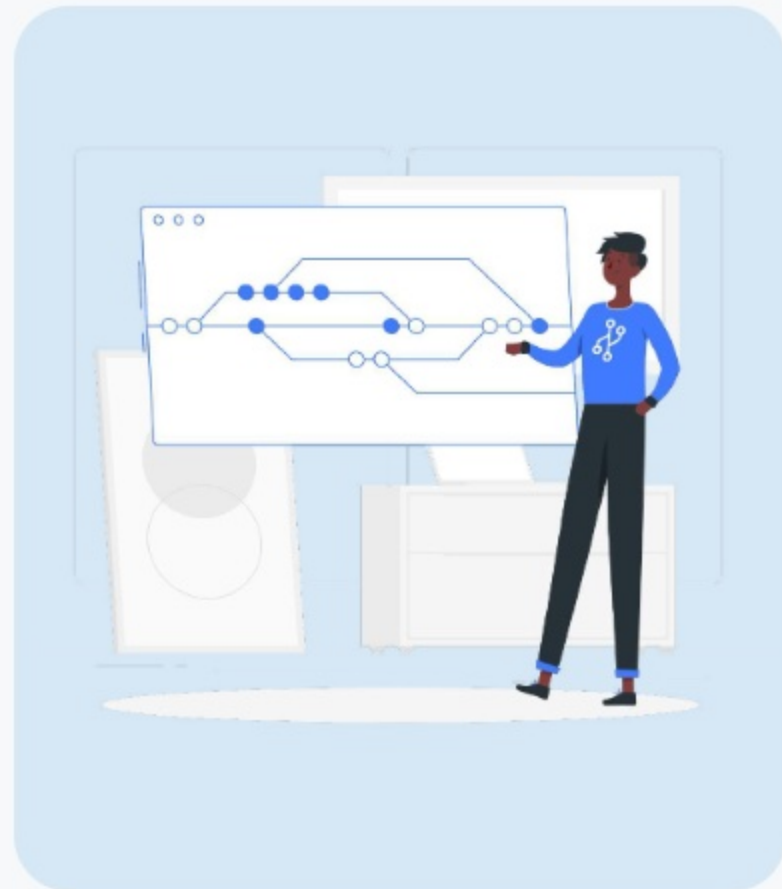
Unlocked packages are **designed for customers and partners** to **organize and deploy metadata** and **business logic** using **modular** development.

❖ USE CASES

Unlocked packages are suitable for **iterative development** and **release of applications** within **customers orgs** or **distributing apps** not intended for AppExchange. It can be used to **organize existing metadata**, **package** an app, **extend** an AppExchange app, or **package** new metadata.

❖ MODULAR DEVELOPMENT

Unlocked packages facilitate modular app development, which provides benefits such as **better ownership** of functionality, more efficient **change management**, more efficient **development processes**, and **decreased cost** to deliver new features.



Unlocked Packages

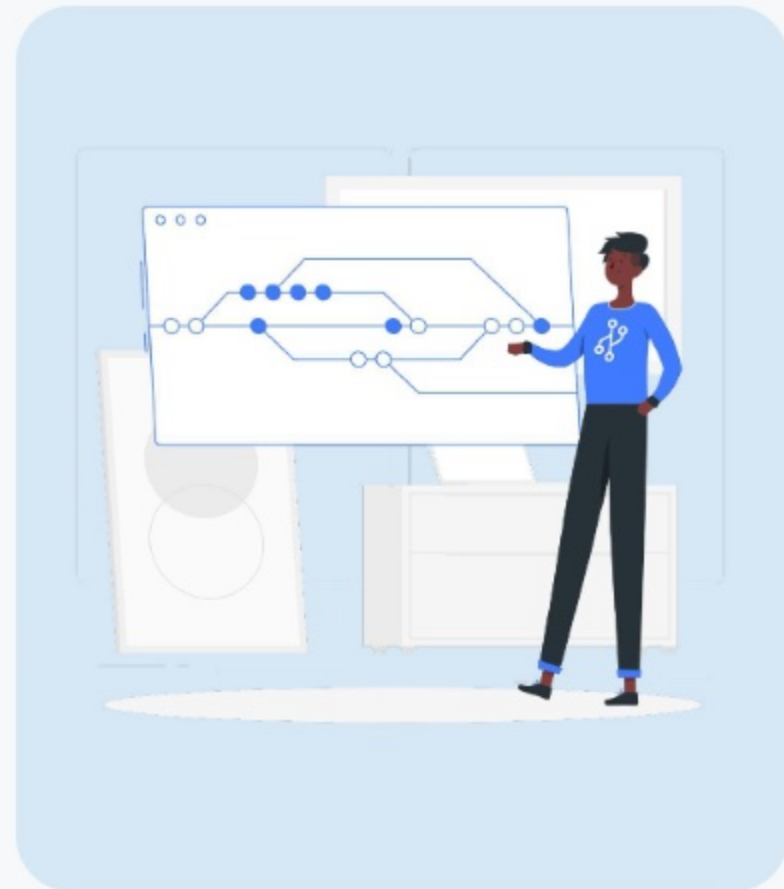
Unlocked packages are **designed for customers and partners** to **organize and deploy metadata** and **business logic** using **modular** development.

❖ SOURCE-DRIVEN DEVELOPMENT

Unlocked packages utilize a **source-driven development model**. The **source of truth** of the metadata contained in an unlocked package is the **version control system** rather than what's in the org.

❖ DEPENDENCIES

Unlocked packages allow declaring **multi-level dependencies** on one or many **managed** and **unlocked packages**.



Unlocked Packages

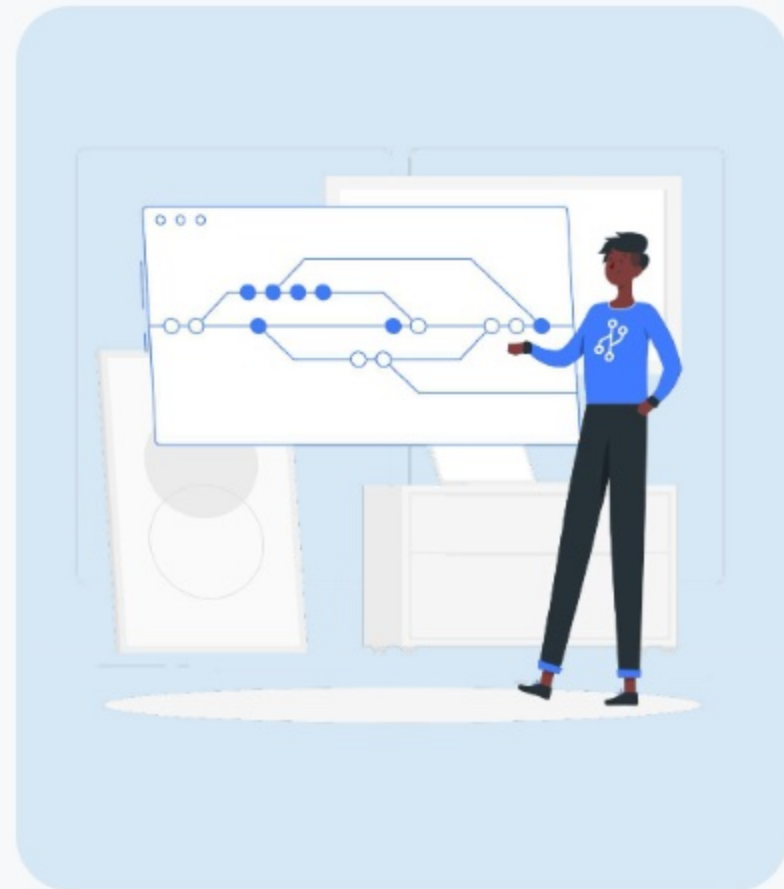
Unlocked packages are **designed for customers and partners** to **organize and deploy metadata** and **business logic** using **modular** development.

❖ PACKAGE EVOLUTION

An unlocked package has a distinct life cycle. While a package **continually evolves**, each **package version** is an **immutable** artifact that contains **specific metadata and features** associated it at the time it was created. As a developer **adds**, **removes**, or **changes** the packaged metadata, they create many package versions.

❖ INSTALLATION

A **package version** can be installed in a scratch, sandbox, trial, developer edition, or production org. It has a **version number**, and subscribers can install a new package version through a **package upgrade**.



Unlocked Packages

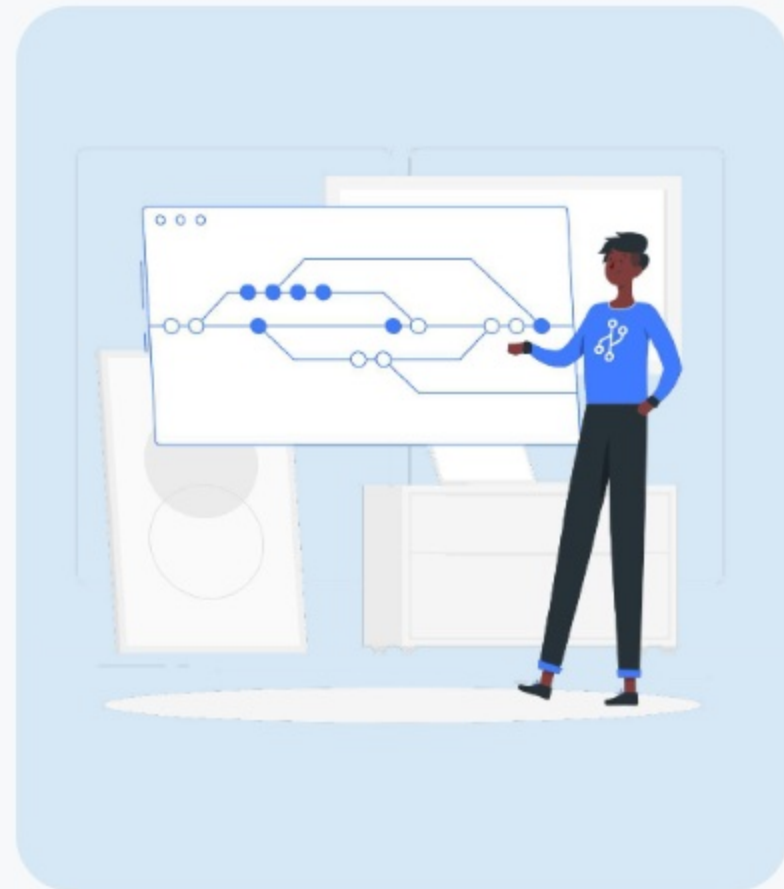
Unlocked packages are **designed for customers and partners** to **organize and deploy metadata** and **business logic** using **modular** development.

◆ NAMESPACE

An unlocked package can include a **namespace** for **isolation** or be **namespace-free** for **more seamless integration** with org metadata.

◆ REQUIREMENTS

Development of unlocked packages requires enabling **Dev Hub** and **Second-Generation Managed Packaging** and installing **Salesforce CLI**. A **Salesforce DX** project is created for development.

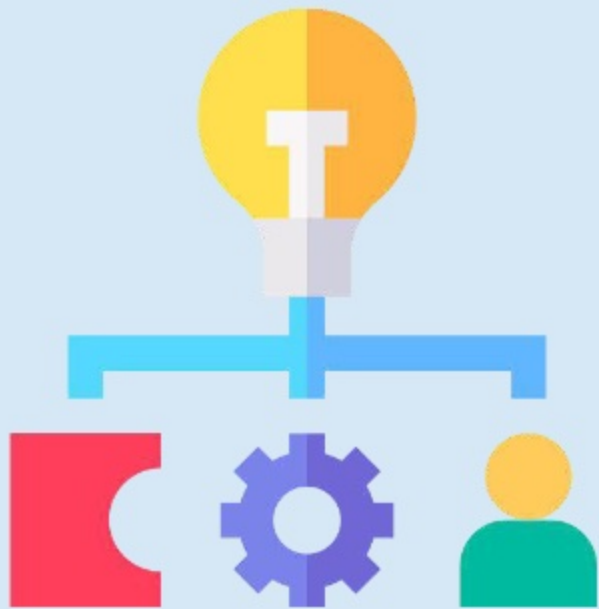


Business Scenario

Table of Contents 

Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Cosmic Learning Solutions, a leader in educational technology, utilizes Salesforce to manage information about training modules and enrolled students. In order to enhance learning experiences, promote its products, and streamline administrative tasks across its global education centers, the company wants to build scalable Salesforce solutions using a modular development approach.

The company's Salesforce architect must recommend package development strategies for various requirements, considering implications related to aspects such as IP protection, upgradeability, and collaboration within the educational technology sector. Each strategy should cover the use cases and considerations related to the use of managed, unmanaged, or unlocked packages, based on the company's unique requirements.

Requirements & Solutions

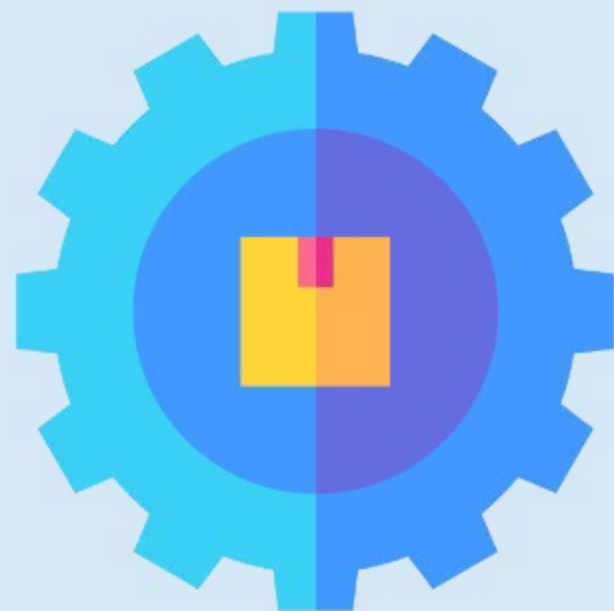
[Table of Contents](#) 

Requirement 1



SCENARIO

The company has developed a managed package for AppExchange. It includes Apex classes to create and customize some metadata components. The package is in the testing phase and has not been certified yet. During testing in a sandbox, the Apex code fails. The architect needs to ensure that the Apex classes are able to access the metadata in the org. Once the testing is complete, the package will be submitted for Salesforce security review. However, the product manager is concerned about the errors from the Checkmarx code scanner and requires a suitable approach to proceed.



Solution 1



SOLUTION

To ensure that Apex classes in the managed package are able to access the metadata in the sandbox to perform operations, **Deploy Metadata from Non-Certified Package Versions via Apex** should be enabled on the **Apex Settings** page in Setup. With regard to the errors returned by the **Checkmarx code scanner**, they should be **reviewed** thoroughly. If there is no need to fix them, they should be marked as **false positive**, an explanation should be attached, and then the package should be submitted for review.

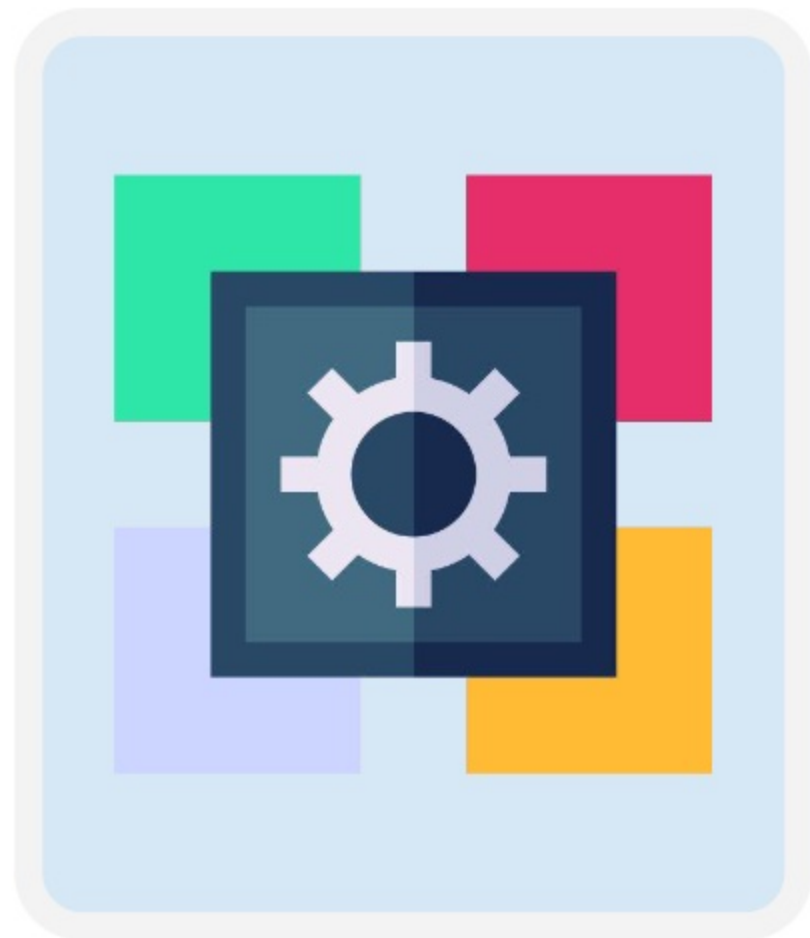


Requirement 2



SCENARIO

Two development teams have started working on two separate projects. Team A is working on an application for learning management, while Team B is working on an admin console for managing various administrative tasks. Both teams will be using second-generation managed packages (managed 2GPs) for development. Team A needs to use certain Apex code that will be developed by Team B. The architect must ensure that Team A only uses the methods exposed by Team B and no other team is able to use the same Apex code.



Solution 2



SOLUTION

Two second-generation packages should be created with the **same namespace** for the given requirements. In the package being developed by team B, the **@namespaceAccessible** annotation should be added to the Apex methods that should be shared with team A.

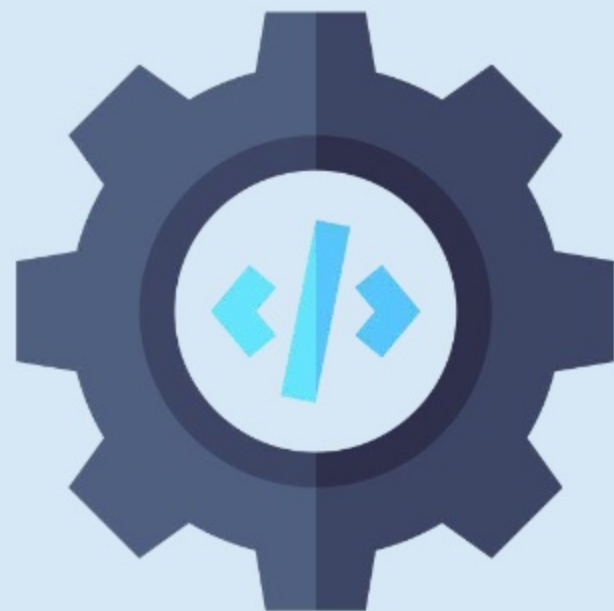


Requirement 3



SCENARIO

The company recently acquired a small business named Northern Lights Solutions, which became a Salesforce partner not long ago. Its development team needs to work on an AppExchange solution and would like the architect to recommend suitable development environments for creating a managed package for the solution. Also, the sales leadership team of Cosmic Learning Solutions recently had hands-on experience with a sales analytics app used by Northern Lights Solutions and would like to add it to the main Salesforce org in the shortest possible time.

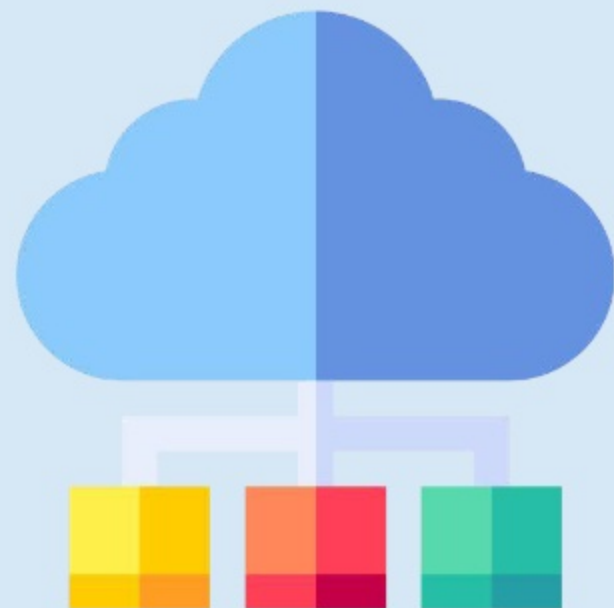


Solution 3



SOLUTION

Northern Lights Solutions' development team can use a **Developer Edition** or **Partner Developer Edition** org to develop the managed package. An **unmanaged package** can be created for the sales analytics app used by Northern Lights Solutions and deployed in the main Salesforce org used by Cosmic Learning Solutions.



Requirement 4



SCENARIO

The company has three orgs in different regions. Its business processes are standardized but used globally, but each region needs the flexibility to be able to understand the standardized code and customize some of its aspects for the region. The architect needs to suggest a suitable package development model for this use case and provide best practices and considerations for change management.

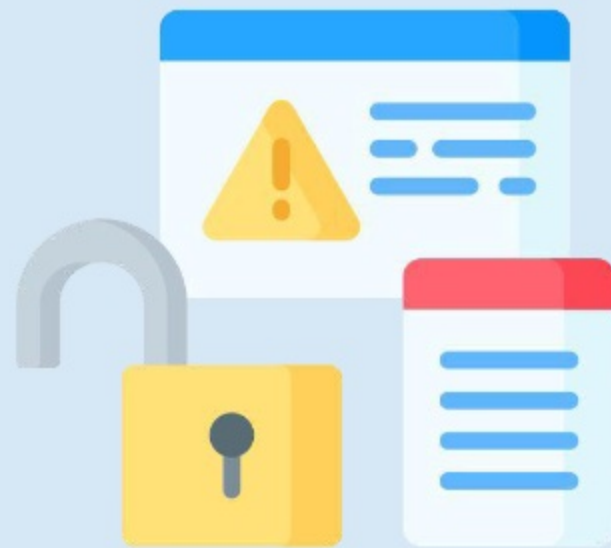


Solution 4



SOLUTION

Unlocked packages can be used to deploy the global code to different Salesforce orgs. Each region can be allowed to create its **customized unlocked package extensions**. For an **agile and efficient release cycle**, a **modular development process** should be utilized with a **source control system** as the source of truth for the metadata in unlocked packages. Developed packages should be tested in sandbox environments before installing to production. The **metadata coverage report** should be consulted to identify features supported by packages. **Package versioning** and the ability to **specify multi-level dependencies** should help with change management.



Requirement 5



SCENARIO

The company wants to release an application that can be installed in partner Salesforce environments. The application is designed to allow the partners to publish training modules without leaving their Salesforce environment. The partners should be able to build on top of the application with flows and Apex triggers for integration with their own business processes. The architect needs to recommend a suitable mechanism for developing and releasing the application, considering the company wants to keep the application up to date in various environments.

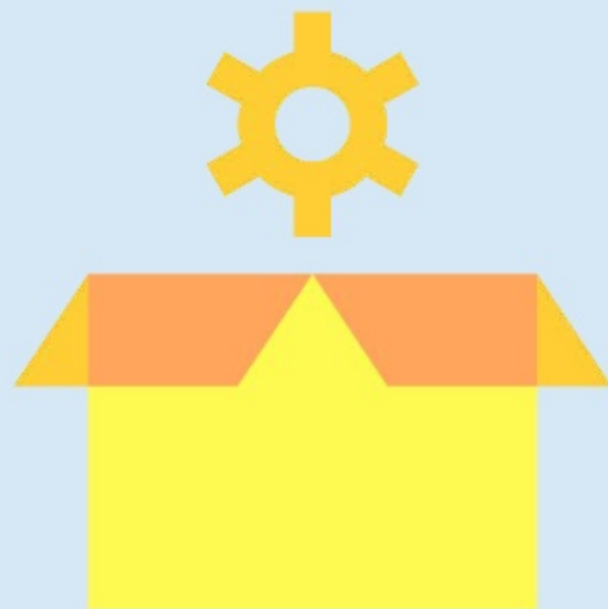


Solution 5



SOLUTION

The company should use a **managed package** for developing and releasing the application. Managed packages allow for **version control**, **easy updates**, and **package upgrades**, which are crucial for keeping the application up to date across partner Salesforce environments. Additionally, managed packages provide **IP protection** and **namespace isolation**, which helps prevent conflicts with the partners' existing customizations. Partners can still build on top of the application with flows and Apex triggers, making this a seamless and scalable solution for distribution.



Requirement 6



SCENARIO

A development team is using two second-generation managed packages for developing a custom application for managing opportunities. Both packages have the same namespace. Package A contains two Apex classes named `OpportunityController` and `OpportunityControllerTest`, but they are used only in package B. The architect needs to recommend a suitable approach for moving these classes to package B for better organization and control.

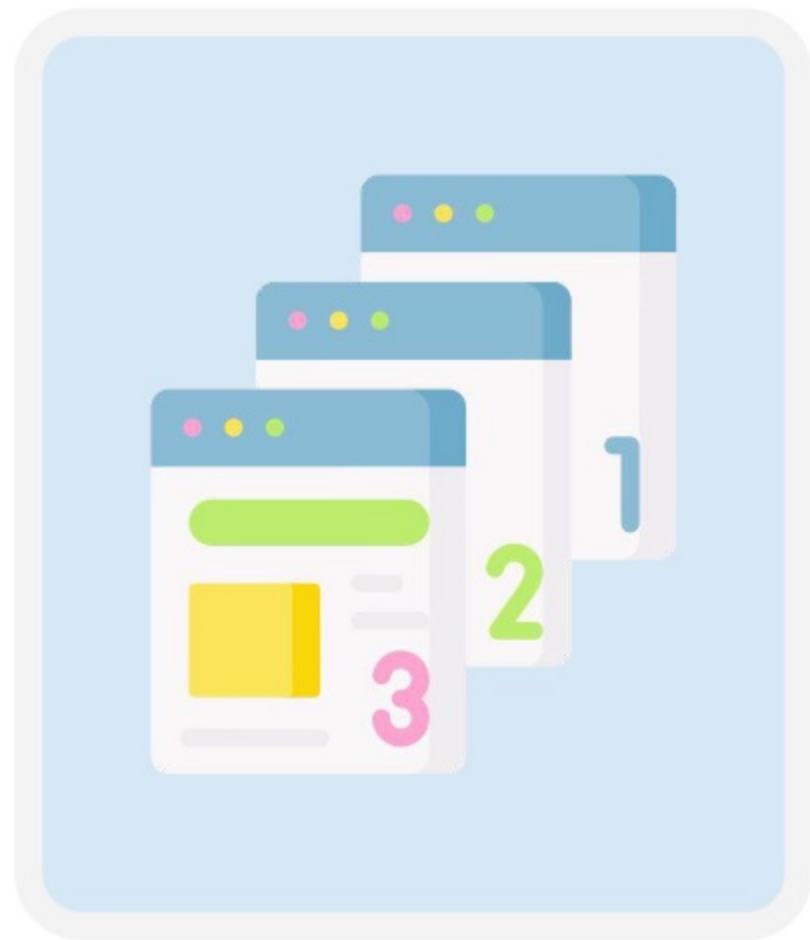


Solution 6



SOLUTION

In package B, any **references to the Apex classes** in package A should be **removed**, and a **new package version** should be created for package B. After removing all the references, the **Apex classes should be removed** from package A, and a **new version of package A** should be created.



Learn More

- [!\[\]\(1207edb9a08751d3d55970560645ed23_img.jpg\) Second-Generation Managed Packages](#)
- [!\[\]\(d7a34a706cfa4ef37c62a369101e1b36_img.jpg\) Second-Generation Managed Packaging Developer Guide](#)
- [!\[\]\(7325769475e8f4bf67f57a0cbebc8ab9_img.jpg\) Developing and Distributing Unmanaged Packages](#)
- [!\[\]\(1a468f12cdfc63dc07896d0781cf55ec_img.jpg\) Unlocked Packages](#)
- [!\[\]\(a9a0baec8ceb7d7c04180806eca8d32a_img.jpg\) Prepare for the Security Review](#)
- [!\[\]\(c1ab807d6aebb565b3082513037b5622_img.jpg\) Security Considerations](#)

